

TechTribers

Java Interview Series · Part 3 · Q122–Q170

Java Platform Interview Questions

Collections, generics, I/O, concurrency, JDBC, Java 8 streams, design patterns and testing, answered the way a senior engineer would answer them.

49	46	30	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 `JP-007`

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**

A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**

What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**

The next question the interviewer is likely to ask, with the answer.

49	9	46	30
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

📦	01	Arrays, Packages & Access Control	Q122–Q124	5
💾	02	Java I/O & Serialization	Q125–Q129	8
🧬	03	Generics	Q130–Q134	12
🧺	04	Collections Framework	Q135–Q141	16
🧵	05	Concurrency in Practice	Q142–Q145	23
🔌	06	Networking, JDBC & Runtime Loading	Q146–Q151	28
λ	07	Java 8 Functional Programming	Q152–Q154	34
🧭	08	Design Patterns	Q155–Q164	37
✅	09	Best Practices, Testing & Debugging	Q165–Q170	47

© TechTribers. For personal use only. Do not redistribute.

All questions

📦 Arrays, Packages & Access Control

Q122 What is an array in Java? How do you declare and use one? 5

Q123 What are packages in Java and why are they used? 6

Q124 What are the access modifiers in Java? 7

💾 Java I/O & Serialization

Q125 What is the difference between byte streams and character streams? 8

Q126 How do you read from and write to files in Java? 9

Q127 What is the difference between `java.io.File` and `java.nio.file.Path` / `Files`? 10

Q128 What is serialization in Java, and how does it work? 10

Q129 What are `serialVersionUID` and `transient`? 11

🧬 Generics

Q130 What are generics and why are they used? 12

Q131 How do you create a generic class? 13

Q132 What is a generic method? 14

Q133 What are wildcards in generics? Explain `? extends` and `? super`. 14

Q134 What is type erasure? 15

🧺 Collections Framework

Q135 What is the Collections Framework? Explain List, Set and Map. 16

Q136 What is the difference between ArrayList and LinkedList? 18

Q137 What is the difference between HashSet, LinkedHashSet and TreeSet? 19

Q138 What is the difference between HashMap, LinkedHashMap and TreeMap? 20

Q139 What is an Iterator? What are fail-fast and fail-safe iterators? 20

Q140 How do you sort and search collections in Java? 21

Q141 What is the difference between Comparable and Comparator? 22

🧵 Concurrency in Practice

Q142 What is synchronization? How does the `synchronized` keyword work? 23

Q143 What is thread safety? How do you achieve it? 24

Q144 How do threads communicate? Explain `wait()`, `notify()` and `notifyAll()`. 25

Q145 What is a thread pool? How do you use ExecutorService? 27

🔌 Networking, JDBC & Runtime Loading

Q146 What is the difference between TCP and UDP? How do you use UDP in Java? 28

Q147 What is JDBC? How do you connect to a database? 29

Q148 What is the difference between Statement and PreparedStatement? 30

Q149 How do you manage transactions in JDBC? 31

Q150 What are layout managers in Swing? 32

Q151 What is dynamic class loading? How does `Class.forName()` work? 33

λ Java 8 Functional Programming

Q152 What are lambda expressions? 34

Q153 What are functional interfaces? Name the common ones. 35

Q154 What is the Stream API? How do you use it? 36

🧭 Design Patterns

Q155 What are design patterns? What are the main categories? 37

Q156 What is the Singleton pattern? How do you implement it correctly? 38

Q157 What is the Factory pattern? Factory Method vs Abstract Factory? 39

Q158 What is the Builder pattern and when should you use it? 40

Q159 What is the Adapter pattern? 41

Q160 What is the Decorator pattern? 42

Q161 What is the Proxy pattern? 43

Q162 What is the Observer pattern? 44

Q163 What is the Strategy pattern? 45

Q164 What is the Template Method pattern? 46

© TechTribers. For personal use only. Do not redistribute.

✅ Best Practices, Testing & Debugging

Q165	What are Java naming conventions and code-formatting best practices?	47
Q166	What are the best practices for exception handling?	48
Q167	How do memory leaks happen in Java, and how do you prevent them?	48
Q168	How do you approach Java performance optimization?	49
Q169	How do you write unit tests with JUnit 5?	50
Q170	What debugging techniques and tools do you use?	51

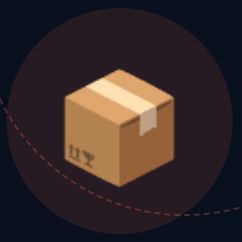
© TechTribers. For personal use only. Do not redistribute.

Section 01

Arrays, Packages & Access Control

Fixed-size arrays, how packages organise code, and the four access levels that decide who can see what.

Questions 122–124 · 3 questions

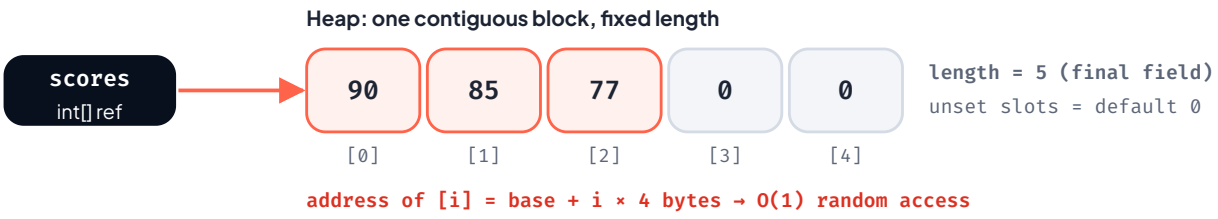


Q122 JP-001 • Basic

What is an array in Java? How do you declare and use one?

⚡ Quick answer

An array is a fixed-length, indexed container of elements of one type. Arrays are objects on the heap, have a final `length` field, and give O(1) access by index.



Java

```java
int[] scores = new int[5];                  // all 0 by default
scores[0] = 90; scores[1] = 85; scores[2] = 77;

String[] names = {"Asha", "Ravi", "Om"};    // initializer
int[][] grid = new int[3][4];               // 2D: array of arrays

System.out.println(scores.length + " " + Arrays.toString(scores));
Arrays.sort(names);
System.out.println(Arrays.toString(names) + " " + Arrays.binarySearch(names, "Om"));
int[] bigger = Arrays.copyOf(scores, 8);    // "resize" = copy into a new array
System.out.println(bigger.length + " " + Arrays.stream(scores).sum());
```

▶ Output

```
5 [90, 85, 77, 0, 0]
[Asha, Om, Ravi] 1
8 252
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Arrays never grow. Index `length` or beyond throws `ArrayIndexOutOfBoundsException`, and `Arrays.asList(intArray)` gives a `List<int[]>` with one element, not a list of numbers.

🎯 **Likely follow-up**

Array or `ArrayList`? Arrays for fixed-size primitive data where performance matters (no boxing); `ArrayList` for everything that grows or needs collection APIs.

🗂️ **Q123** `JP-002` • Basic

What are packages in Java and why are they used?

⚡ **Quick answer**

A package is a namespace that groups related types, prevents name clashes, and enables package-level access control. It maps to a directory path on disk.

Typical layered structure: `com.company.app.controller`, `.service`, `.repository`, `.model`. Many teams now prefer **package-by-feature** (`.billing`, `.orders`) because it keeps each feature's classes together and lets you hide internals with package-private access.

Since Java 9, **modules** (`module-info.java`) add a level above packages: a module chooses which packages it `exports`, so even `public` classes in unexported packages are hidden.

Java

```java
// File: src/main/java/com/techtribers/billing/InvoiceService.java
package com.techtribers.billing;                // reverse domain name

import java.time.LocalDate;                     // single-type import
import java.util.*;                             // on-demand import (avoid in prod code)
import static java.util.Objects.requireNonNull; // static import

public class InvoiceService {
    public Invoice create(Customer c) {
        requireNonNull(c);
        return new Invoice(c, LocalDate.now());
    }
}
```

⚠️ **Interview trap**

`import java.util.*` does not import sub-packages like `java.util.concurrent`. Packages are not hierarchical for access purposes: `com.a.b` gets no special access to `com.a`.

© TechTribers. For personal use only. Do not redistribute.

Q124 JP-003

• Basic

What are the access modifiers in Java?

⚡ **Quick answer**

Four levels, from most to least open: `public`, `protected`, package-private (no keyword), and `private`. They control which classes can see a type or member.

Modifier	Same class	Same package	Subclass (other pkg)	Everywhere
`public`	✓	✓	✓	✓
`protected`	✓	✓	✓	✗
(default)	✓	✓	✗	✗
`private`	✓	✗	✗	✗

Java

```java
public class Account {
    private BigDecimal balance;          // only Account
    String branchCode;                   // package-private: same package only
    protected void audit() { }           // package + subclasses
    public BigDecimal balance() { return balance; }
}
```

⚠️ **Interview trap**

Top-level classes can only be `public` or package-private. `private` and `protected` apply to members and nested classes only.

💡 **Pro tip**

Default to the most restrictive level that works: `private` fields, package-private helpers, `public` only for the real API. Every `public` member is a promise you must keep forever.

© TechTribers. For personal use only. Do not redistribute.

Section 02

Java I/O & Serialization

Streams, readers and writers, modern file APIs, and turning objects into bytes safely.

Questions 125–129 · 5 questions



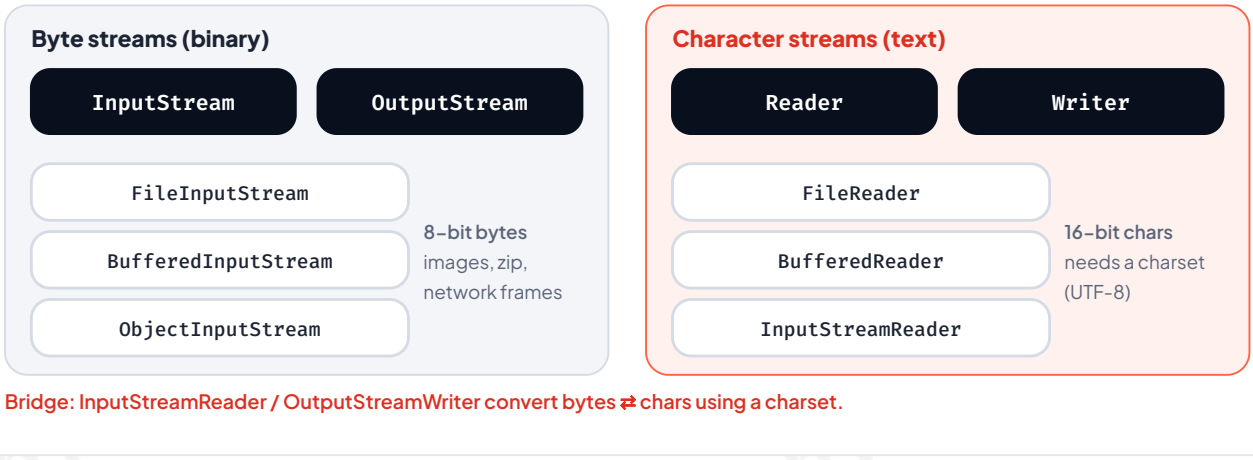
Q125 JP-004

• Basic

What is the difference between byte streams and character streams?

⚡ Quick answer

Byte streams (`InputStream` / `OutputStream`) move raw 8-bit bytes and suit binary data. Character streams (`Reader` / `Writer`) move 16-bit chars and decode/encode text using a charset.



Java

```java
// Binary: copy an image byte-for-byte
try (var in = new BufferedInputStream(new FileInputStream("logo.png"));
     var out = new BufferedOutputStream(new FileOutputStream("logo-copy.png"))) {
    in.transferTo(out);                                  // Java 9+
}

// Text: bytes -> chars with an explicit charset
try (var reader = new BufferedReader(
        new InputStreamReader(new FileInputStream("notes.txt"), StandardCharsets.UTF_8))) {
    reader.lines().limit(3).forEach(System.out::println);
}
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Reading a PDF or image through a `Reader` corrupts it, because invalid byte sequences get replaced during decoding. Text → Reader/Writer, everything else → streams.

💡 **Pro tip**

Always wrap with `Buffered*`. Unbuffered `FileInputStream.read()` makes one system call per byte, which can be 100× slower.

Q126 JP-005 • Basic

How do you read from and write to files in Java?

⚡ **Quick answer**

For most cases use `java.nio.file.Files` (Java 7+/11+): `readString`, `writeString`, `readAllLines`, `lines`. Use buffered readers/writers for large files or streaming.

Java

```
Path file = Path.of("output.txt");

Files.writeString(file, "Hello, World!\nThis is a sample file.\n");          // Java 11
Files.writeString(file, "Appended line\n", StandardOpenOption.APPEND);

String all = Files.readString(file);                                        // small files
System.out.print(all);

try (Stream<String> lines = Files.lines(file)) {                            // large files: lazy
    System.out.println("Lines: " + lines.count());
}

try (BufferedWriter w = Files.newBufferedWriter(Path.of("big.csv"))) {      // UTF-8 by default
    for (int i = 1; i <= 3; i++) { w.write("row," + i); w.newLine(); }
}
```

▶ **Output**

```
Hello, World!
This is a sample file.
Appended line
Lines: 3
```

⚠️ **Interview trap**

`Files.lines()` keeps the file open until the stream is closed, so always use it in try-with-resources. And `readAllLines()` on a multi-GB log loads everything into heap and causes `OutOfMemoryError`.

© TechTribers. For personal use only. Do not redistribute.

Q127 JP-006

• Intermediate

What is the difference between `java.io.File` and `java.nio.file.Path` / `Files`?

⚡ Quick answer

`File` (Java 1.0) is the legacy API: it returns booleans on failure and lacks proper error detail. `Path` + `Files` (NIO.2, Java 7) throw descriptive exceptions and support symlinks, attributes, walking trees and file watching.

	java.io.File	java.nio.file Path + Files
Errors	`delete()` returns `false`, no reason	Throws `NoSuchFileException`, `AccessDeniedException` …
Copy / move	Not available	`Files.copy`, `Files.move` (atomic option)
Directory walk	Manual recursion	`Files.walk`, `Files.find`
Metadata	Basic	Permissions, owner, symlinks, file stores
Watch for changes	No	`WatchService`

Java

```
Path dir = Path.of("reports");
Files.createDirectories(dir);
Files.copy(Path.of("q3.pdf"), dir.resolve("q3.pdf"), StandardCopyOption.REPLACE_EXISTING);

try (Stream<Path> pdfs = Files.walk(dir)) {
    pdfs.filter(p -> p.toString().endsWith(".pdf")).forEach(System.out::println);
}
File legacy = dir.toFile();          // convert when an old API needs File
Path back = legacy.toPath();
```

Q128 JP-007

• Intermediate

What is serialization in Java, and how does it work?

⚡ Quick answer

Serialization converts an object graph into a byte stream (`ObjectOutputStream.writeObject`) and deserialization rebuilds it (`ObjectInputStream.readObject`). A class opts in by implementing the `Serializable` marker interface.

Rules: every non-transient field must itself be serializable (or you get `NotSerializableException`), and deserialization does **not** run the class's constructor, except for records, which are rebuilt through their canonical constructor.

© TechTribers. For personal use only. Do not redistribute.



transient and static fields are skipped. serialVersionUID must match on read, or InvalidClassException.

Java

```java
record Student(String name, int age, String major) implements Serializable { }

try (var out = new ObjectOutputStream(Files.newOutputStream(Path.of("data.ser")))) {
    out.writeObject(new Student("Asha", 22, "CS"));
}
try (var in = new ObjectInputStream(Files.newInputStream(Path.of("data.ser")))) {
    Student s = (Student) in.readObject();
    System.out.println(s);
}
```

▶ Output

```
Student[name=Asha, age=22, major=CS]
```

⚠️ **Interview trap**

Never deserialize untrusted data with `ObjectInputStream`. "Deserialization gadget chains" have caused remote-code-execution bugs in many Java servers. Use JSON/Protobuf for data exchange, or configure an `ObjectInputFilter` (Java 9+).

🏷️ **Q129** JP-008 • Intermediate

What are `serialVersionUID` and `transient`?

⚡ **Quick answer**

`serialVersionUID` is the class's serialization version; if the stored and current values differ, deserialization fails with `InvalidClassException`. `transient` marks a field to be skipped during serialization.

Java

```java
public class UserSession implements Serializable {
    @Serial
    private static final long serialVersionUID = 1L;    // declare it explicitly

    private String userId;
    private transient String password;                  // never written to disk
    private transient Connection db;                    // not serializable anyway
```


© TechTribers. For personal use only. Do not redistribute.

```
    // after deserialization: password == null, db == null
}
```

⚠️ **Interview trap**

If you don't declare `serialVersionUID`, the JVM computes one from the class structure. Adding a single method then changes it, and every previously saved object fails to load.

💡 **Pro tip**

`transient` fields come back as default values (`null`, `0`, `false`). Re-initialise them in a `readObject` method or lazily on first use.

Section 03

Generics

Type-safe reusable code: generic classes and methods, wildcards and the PECS rule, and what type erasure really means.

Questions 130–134 · 5 questions



Q130 JP-009 • Basic

What are generics and why are they used?

⚡ **Quick answer**

Generics (Java 5) let classes, interfaces and methods take **type parameters**, so the compiler checks element types and you don't need casts. Errors move from runtime (`ClassCastException`) to compile time.

Benefits to name: **type safety**, **no casts**, and **reusable algorithms** (one `sort` for any `Comparable` type).

Java

```
// Before generics: anything goes, fails at runtime
List raw = new ArrayList();
raw.add("Asha");
raw.add(42);
// String s = (String) raw.get(1);        // ClassCastException at runtime

// With generics: the compiler refuses bad code
List<String> names = new ArrayList<>();     // diamond operator (Java 7)
names.add("Asha");
// names.add(42);                          // compile error
```


© TechTribers. For personal use only. Do not redistribute.

```
String first = names.get(0);                // no cast needed
System.out.println(first.toUpperCase());
```

▶ Output

```
ASHA
```

⚠️ Interview trap

Generics work only with reference types: `List<int>` is illegal, so you write `List<Integer>` and pay for boxing. Use `IntStream` or primitive arrays for hot numeric code.

📦 Q131 JP-010 • Basic

How do you create a generic class?

⚡ Quick answer

Declare type parameters in angle brackets after the class name (`class Box<T>`) and use `T` like a normal type inside. Callers choose the actual type: `Box<Integer>` .

Naming convention: `T` type, `E` element, `K` / `V` key/value, `R` result, `N` number. **Bounded** type parameters restrict what callers can pass: `class Stats<N extends Number>` lets you call `n.doubleValue()` inside.

Java

```
public class Box<T> {
    private final T value;
    public Box(T value) { this.value = value; }
    public T get() { return value; }
    public <R> Box<R> map(Function<? super T, ? extends R> f) {   // generic method inside
        return new Box<>(f.apply(value));
    }
}

record Pair<K, V>(K key, V value) { }                          // multiple params

Box<Integer> age = new Box<>(30);
Box<String> label = age.map(a -> "Age: " + a);
System.out.println(label.get() + " | " + new Pair<>("lang", "Java"));
```

▶ Output

```
Age: 30 | Pair[key=lang, value=Java]
```


© TechTribers. For personal use only. Do not redistribute.

⚙️ Q132 JP-011

• Intermediate

What is a generic method?

⚡ Quick answer

A method that declares its own type parameters before the return type: `static <T> T first(List<T> list)`. The compiler infers `T` from the arguments at each call.

Java

```
public final class Utils {
    public static <T> Optional<T> first(List<T> list) {
        return list.isEmpty() ? Optional.empty() : Optional.of(list.get(0));
    }
    public static <T extends Comparable<? super T>> T max(List<T> items) {   // bounded
        T best = items.get(0);
        for (T it : items) if (it.compareTo(best) > 0) best = it;
        return best;
    }
}

System.out.println(Utils.first(List.of("Asha", "Ravi")).orElse("none"));
System.out.println(Utils.max(List.of(3, 42, 7)));
System.out.println(Utils.<String>first(List.of()).orElse("none"));   // explicit type witness
```

▶ Output

```
Asha
42
none
```

⚠️ Interview trap

Returning `null` from a generic "find" method (as many tutorials do) pushes NPEs onto every caller. Return `Optional<T>`.

❓ Q133 JP-012

• Advanced

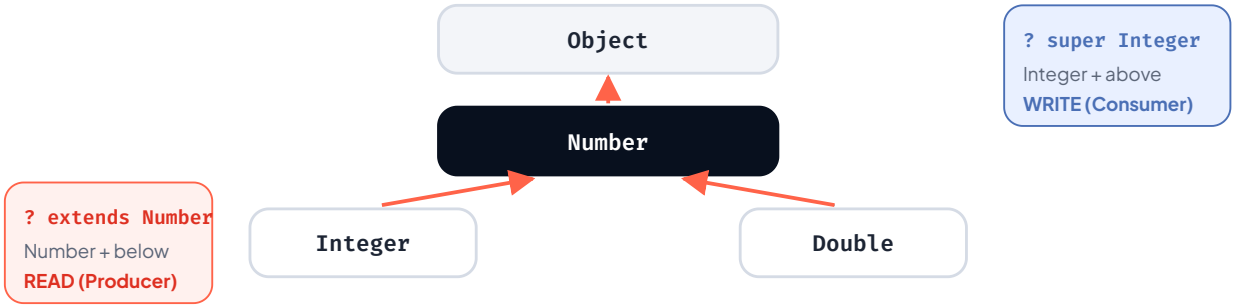
What are wildcards in generics? Explain `? extends` and `? super`.

⚡ Quick answer

Wildcards make generic parameters flexible. `? extends T` (upper bound) accepts T or subtypes and is for **reading**. `? super T` (lower bound) accepts T or supertypes and is for **writing**. Plain `?` means any type.

Why it's needed: `List<Integer>` is **not** a subtype of `List<Number>` (generics are invariant). Without wildcards, `sum(List<Number>)` would refuse a `List<Integer>`.

© TechTribers. For personal use only. Do not redistribute.



PECS: Producer Extends, Consumer Super. Plain ? = read as Object only.

```java
static double sum(List<? extends Number> nums) {          // producer: read only
    double total = 0;
    for (Number n : nums) total += n.doubleValue();
    // nums.add(1);                                        // compile error
    return total;
}

static void addDefaults(List<? super Integer> sink) {      // consumer: write
    sink.add(0); sink.add(1);
}

List<Number> mixed = new ArrayList<>(List.of(2.5));
addDefaults(mixed);                                        // List<Number> is a super of Integer
System.out.println(sum(List.of(1, 2, 3)) + " " + sum(mixed) + " " + mixed);
```

▶ Output

```
6.0 3.5 [2.5, 0, 1]
```

⚠️ **Interview trap**

A common wrong statement (it appears in many notes): "`<?>` is the upper-bounded wildcard." No: `<?>` is the **unbounded** wildcard, `<? extends T>` is upper-bounded, and `<? super T>` is lower-bounded.

Q134 JP-013 • Advanced

What is type erasure?

⚡ **Quick answer**

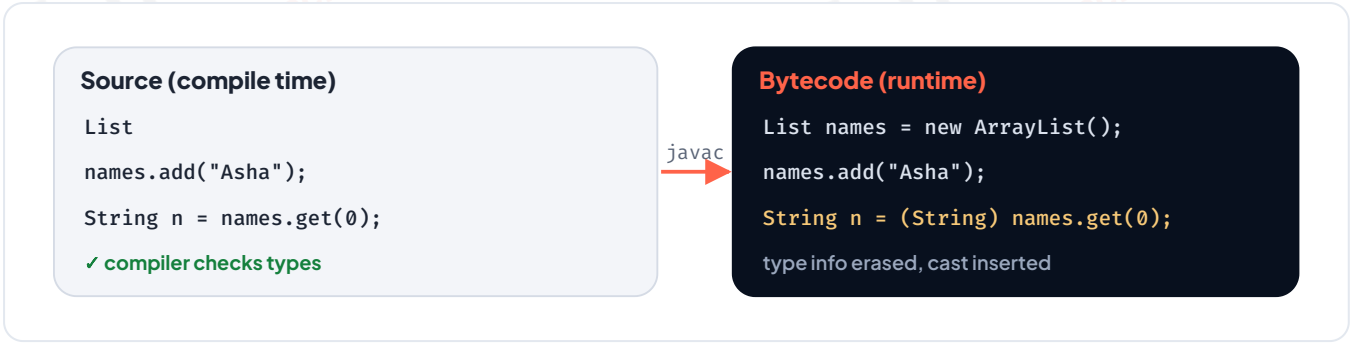
The compiler checks generic types and then **erases** them: type parameters become their bound (`Object` by default) in bytecode, and casts are inserted. At runtime `List<String>` and `List<Integer>` are the same class.

Erasure was chosen in Java 5 so generic code could run on old JVMs and interoperate with pre-generic libraries. The workaround when you need the type at runtime is to pass a `Class<T>` token, as

© TechTribers. For personal use only. Do not redistribute.

`EntityManager.find(User.class, id)` and Jackson's `TypeReference` do.

* `instanceof List<?>` (unbounded) is allowed.



Java

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());     // true: both ArrayList

// Consequences of erasure (all compile errors):
// if (a instanceof List<String>) { }                 // can't test parameterized type*
// T obj = new T();                                   // can't instantiate T
// T[] arr = new T[10];                               // can't create generic array
// void f(List<String> x) {}  void f(List<Integer> x) {}  // same erasure: clash
```

▶ Output

```
true
```

Section 04

Collections Framework

Lists, sets, maps and queues: which implementation to pick, how iteration fails fast, and how to sort and search.

Questions 135–141 · 7 questions



Q135 JP-014

• Basic

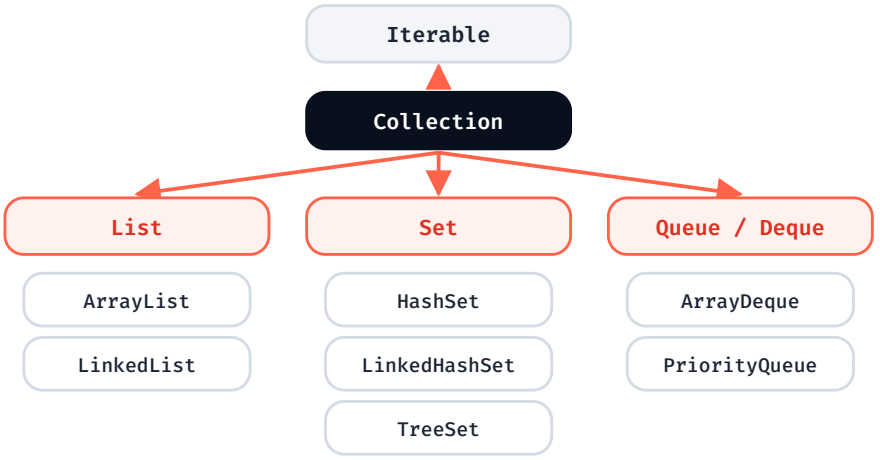
What is the Collections Framework? Explain List, Set and Map.

⚡ Quick answer

A unified set of interfaces (`List` , `Set` , `Queue` , `Map`), implementations and algorithms (`Collections`) for storing groups of objects. List = ordered with duplicates, Set = unique elements, Map = unique keys to values.

© TechTribers. For personal use only. Do not redistribute.

Interface	Order	Duplicates	Go-to implementation
`List`	Insertion / index	Yes	`ArrayList`
`Set`	Depends on impl	No	`HashSet`
`Queue` / `Deque`	FIFO / priority	Yes	`ArrayDeque`
`Map`	Depends on impl	Unique keys	`HashMap`



Java

```java
List<String> names = new ArrayList<>(List.of("Asha", "Ravi", "Asha"));
Set<String> unique = new LinkedHashSet<>(names);      // de-duplicate, keep order
Map<String, Integer> ages = new HashMap<>();
ages.put("Asha", 25);
ages.merge("Ravi", 30, Integer::sum);

System.out.println(names + " | " + unique + " | " + ages.getOrDefault("Om", 0));
```

▶ Output

```
[Asha, Ravi, Asha] | [Asha, Ravi] | 0
```

⚠ **Interview trap**

`Map` does **not** extend `Collection`, and `List.of(...)` / `Map.of(...)` (Java 9) return **immutable** collections that throw `UnsupportedOperationException` on `add` / `put` and reject `null`.

© TechTribers. For personal use only. Do not redistribute.

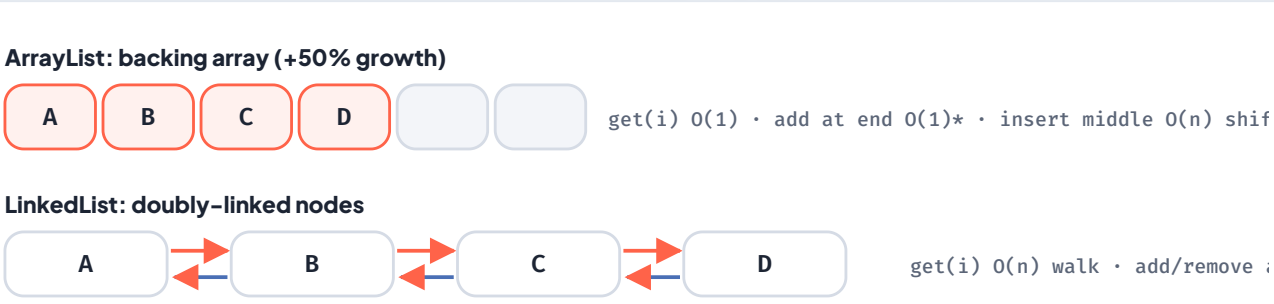
Q136 JP-015 • Basic

What is the difference between ArrayList and LinkedList?

⚡ **Quick answer**

`ArrayList` is backed by a resizable array: fast random access, slower middle inserts. `LinkedList` is a doubly-linked list: O(1) insert/remove at the ends, O(n) access by index.

Operation	ArrayList	LinkedList
`get(i)`	O(1)	O(n)
`add(e)` at end	O(1) amortised	O(1)
`add(0, e)` / remove first	O(n) shift	O(1)
Memory per element	~4–8 bytes reference	~24+ bytes node overhead
Also implements	`RandomAccess`	`Deque`



* amortised. In practice ArrayList wins almost every benchmark thanks to CPU cache locality.

💡 **Pro tip**

Use `ArrayList` by default. If you need a queue or stack, use `ArrayDeque`, which beats `LinkedList` for both. Real-world benchmarks rarely favour `LinkedList` because pointer-chasing defeats the CPU cache.

🎯 **Likely follow-up**

How does ArrayList grow? When full, it allocates a new array about 1.5× larger (`old + old >> 1`) and copies the elements. Pre-size with `new ArrayList<>(expected)` to avoid repeated copies.

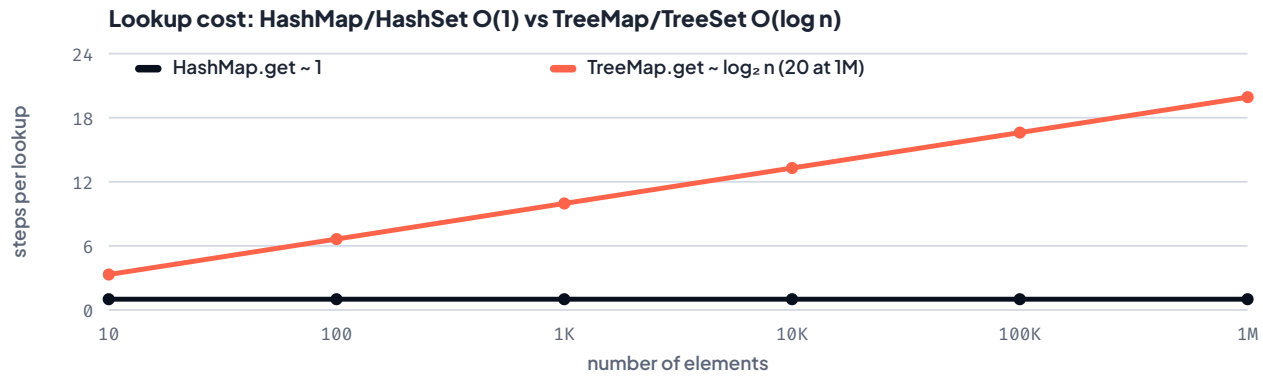
© TechTribers. For personal use only. Do not redistribute.

🎯 **Q137** JP-016 • Intermediate

What is the difference between HashSet, LinkedHashSet and TreeSet?

> ⚡ **Quick answer**
> All three store unique elements. `HashSet` has no order and O(1) operations. `LinkedHashSet` keeps insertion order. `TreeSet` keeps elements **sorted** (red-black tree) with O(log n) operations and navigation methods.

Uniqueness rules differ: hash sets use `hashCode()` + `equals()`; `TreeSet` uses `compareTo()` / `Comparator` only. If `compareTo` returns 0 for two different objects, `TreeSet` treats them as duplicates.




```java
List<String> in = List.of("kafka", "java", "spring", "java");
System.out.println(new HashSet<>(in));          // order not guaranteed
System.out.println(new LinkedHashSet<>(in));    // insertion order
TreeSet<String> sorted = new TreeSet<>(in);
System.out.println(sorted + " first=" + sorted.first() + " ceiling(k)=" + sorted.ceiling("k"));
```

▶ **Output**

```
[spring, java, kafka]
[kafka, java, spring]
[java, kafka, spring] first=java ceiling(k)=kafka
```

> ⚠️ **Interview trap**
> `HashSet` output order can *look* sorted for small strings or integers. It's an accident of hash values, never something to rely on.

© TechTribers. For personal use only. Do not redistribute.

Q138 JP-017

• Intermediate

What is the difference between HashMap, LinkedHashMap and TreeMap?

⚡ Quick answer

`HashMap` : hash table, no order, O(1) average, one `null` key allowed. `LinkedHashMap` : adds insertion or access order (great for LRU caches). `TreeMap` : sorted by key, O(log n), range queries, no `null` keys.

HashMap internals in one breath: an array of buckets; the key's hash picks a bucket; collisions chain in a list that turns into a tree after 8 entries; the table doubles when size exceeds capacity × 0.75 (load factor).

Java

```
Map<String, Integer> hash = new HashMap<>(Map.of("b", 2, "c", 3, "a", 1));
Map<String, Integer> tree = new TreeMap<>(hash);
System.out.println(tree + " head(c)=" + ((TreeMap<String, Integer>) tree).headMap("c"));

// LRU cache in 6 lines: access-ordered LinkedHashMap
class Lru<K, V> extends LinkedHashMap<K, V> {
    private final int cap;
    Lru(int cap) { super(16, 0.75f, true); this.cap = cap; }
    @Override protected boolean removeEldestEntry(Map.Entry<K, V> e) { return size() > cap; }
}
Lru<String, String> cache = new Lru<>(2);
cache.put("x", "1"); cache.put("y", "2"); cache.get("x"); cache.put("z", "3");
System.out.println(cache.keySet());
```

▶ Output

```
{a=1, b=2, c=3} head(c)={a=1, b=2}
[x, z]
```

⚠ Interview trap

`HashMap` is not thread-safe: concurrent puts can lose entries. Use `ConcurrentHashMap`, not `Collections.synchronizedMap` or the legacy `Hashtable`.

Q139 JP-018

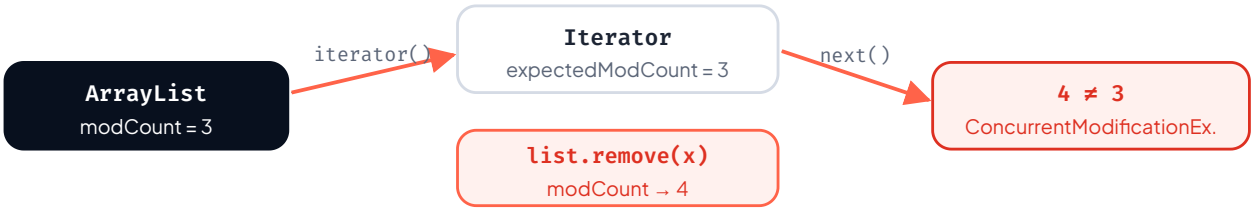
• Intermediate

What is an Iterator? What are fail-fast and fail-safe iterators?

⚡ Quick answer

An `Iterator` walks a collection with `hasNext()` / `next()` and can safely `remove()` the current element. **Fail-fast** iterators throw `ConcurrentModificationException` if the collection is structurally modified during iteration; **fail-safe** ones work on a snapshot or tolerate changes.

© TechTribers. For personal use only. Do not redistribute.



Fail-fast: ArrayList, HashMap. Fail-safe (snapshot/weakly consistent): CopyOnWriteArrayList, ConcurrentHashMap.

Java

```
List<String> langs = new ArrayList<>(List.of("Java", "Cobol", "Kotlin", "Fortran"));

// for (String l : langs) if (l.length() > 6) langs.remove(l);   // ConcurrentModificationException

Iterator<String> it = langs.iterator();
while (it.hasNext()) if (it.next().length() > 6) it.remove();   // safe
System.out.println(langs);

langs.removeIf(l -> l.startsWith("C"));                          // Java 8: simplest
System.out.println(langs);

List<String> cow = new CopyOnWriteArrayList<>(List.of("a", "b"));
for (String s : cow) cow.add("c");                               // fail-safe: iterates snapshot
System.out.println(cow);
```

▶ Output

```
[Java, Cobol, Kotlin]
[Java, Kotlin]
[a, b, c, c]
```

⚠️ **Interview trap**

Fail-fast detection is best-effort, not guaranteed. Never rely on `ConcurrentModificationException` for correctness in multithreaded code; use concurrent collections.

🔍 Q140 JP-019 • Basic

How do you sort and search collections in Java?

⚡ **Quick answer**

Sort with `list.sort(comparator)` or `Collections.sort(list)`; search a **sorted** list with `Collections.binarySearch` in O(log n). Streams offer `sorted()` and `filter().findFirst()`.

Java

```
List<Integer> nums = new ArrayList<>(List.of(5, 3, 1, 4, 2));
Collections.sort(nums);
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(nums + " idx(3)=" + Collections.binarySearch(nums, 3));
System.out.println("idx(10)=" + Collections.binarySearch(nums, 10));     // -(insertion point) - 1

record Emp(String name, String dept, int salary) { }
List<Emp> emps = new ArrayList<>(List.of(new Emp("Asha", "Eng", 90), new Emp("Ravi", "Ops", 70),
                                         new Emp("Om", "Eng", 85)));
emps.sort(Comparator.comparing(Emp::dept).thenComparing(Emp::salary, Comparator.reverseOrder()));
emps.forEach(e -> System.out.print(e.name() + " "));
```

▶ Output

```
[1, 2, 3, 4, 5] idx(3)=2
idx(10)=-6
Asha Om Ravi
```

⚠️ Interview trap

`binarySearch` on an **unsorted** list returns garbage without any error. And a negative result is not "-1 = not found": it encodes where the element would be inserted.

⚖️ **Q141** JP-020 • Intermediate

What is the difference between Comparable and Comparator?

⚡ Quick answer

`Comparable<T>` defines a class's single **natural order** via `compareTo` inside the class. `Comparator<T>` defines any number of **external** orders via `compare`, without touching the class.

	Comparable	Comparator
Package	`java.lang`	`java.util`
Method	`compareTo(T o)`	`compare(T a, T b)`
Where	Inside the class	Outside, often a lambda
How many	One natural order	As many as you need

Java

```
record Product(String name, double price) implements Comparable<Product> {
    @Override public int compareTo(Product o) { return name.compareTo(o.name); }   // natural: by
name
}

List<Product> ps = new ArrayList<>(List.of(new Product("Mouse", 799), new Product("Laptop", 65000),
                                           new Product("Cable", 199)));
Collections.sort(ps);                                                  // uses Comparable
System.out.println(ps.stream().map(Product::name).toList());
```


© TechTribers. For personal use only. Do not redistribute.

```
ps.sort(Comparator.comparingDouble(Product::price).reversed());        // custom order
System.out.println(ps.stream().map(Product::name).toList());
```

▶ Output

```
[Cable, Laptop, Mouse]
[Laptop, Mouse, Cable]
```

⚠️ **Interview trap**

Never write `return a.price - b.price;` in a comparator: integer subtraction overflows for large values and silently breaks sorting. Use `Integer.compare(a, b)` or `Comparator.comparingInt`.

Section 05

Concurrency in Practice

Locks, race conditions, thread communication and thread pools: what goes wrong when threads share data and how to fix it.

Questions 142–145 · 4 questions

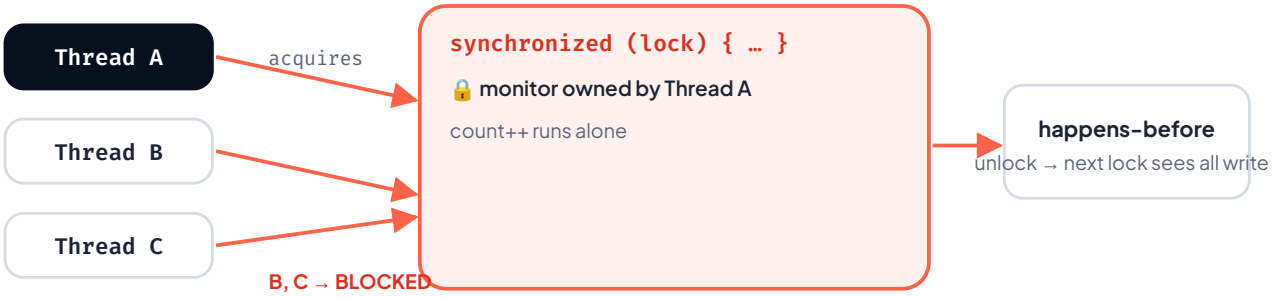


🔒 Q142 JP-021 • Intermediate

What is synchronization? How does the `synchronized` keyword work?

⚡ **Quick answer**

`synchronized` makes a method or block run under an object's **monitor lock**, so only one thread executes it at a time for that lock. It guarantees both mutual exclusion and visibility of changes to the next thread that takes the lock.



Java

```
class Counter {
    private int count;
```


© TechTribers. For personal use only. Do not redistribute.

```
    private final Object lock = new Object();

    public synchronized void increment() { count++; }          // lock = this
    public static synchronized void log() { }                   // lock = Counter.class

    public void add(int n) {
        synchronized (lock) { count += n; }                     // block: smallest critical section
    }
    public synchronized int get() { return count; }
}

Counter c = new Counter();
Thread[] ts = new Thread[4];
for (int i = 0; i < 4; i++) { ts[i] = new Thread(() -> { for (int j = 0; j < 10_000; j++)
c.increment(); }); ts[i].start(); }
for (Thread t : ts) t.join();
System.out.println(c.get());
```

▶ Output

```
40000
```

⚠️ **Interview trap**

The `add()` block locks on `lock` while `increment()` locks on `this`. They don't exclude each other, so mixing them on the same field is a race. All access to one piece of state must use the **same** lock.

💡 **Pro tip**

Synchronized blocks are reentrant: a thread already holding the lock can enter another block on the same lock. For timeouts or try-lock, use `ReentrantLock`.

Q143 JP-022

• Intermediate

What is thread safety? How do you achieve it?

⚡ **Quick answer**

Code is thread-safe when it behaves correctly regardless of how threads interleave. Achieve it with (best first): immutability, thread confinement, thread-safe classes (`Atomic*` , `ConcurrentHashMap`), and locking.

The toolbox:

- * *`volatile`* *: every read sees the latest write; no atomicity, so `volatile int x; x++` is still a race.
- * *`AtomicInteger`* / *`AtomicLong`* / *`LongAdder`* *: atomic updates via compare-and-swap.
- **Concurrent collections**: `ConcurrentHashMap` , `CopyOnWriteArrayList` , `BlockingQueue` .
- **Locks**: `synchronized` , `ReentrantLock` , `ReadWriteLock` , `StampedLock` .

© TechTribers. For personal use only. Do not redistribute.



count++ is read-modify-write. Two increments, final value 6 instead of 7: a lost update.

Java

```
class UnsafeCounter { int count; void inc() { count++; } }            // race condition
class SafeCounter   { final AtomicInteger count = new AtomicInteger();
                      void inc() { count.incrementAndGet(); } }       // lock-free CAS

class Config {
    private volatile boolean reloaded;                                // visibility only
    void markReloaded() { reloaded = true; }
}

UnsafeCounter u = new UnsafeCounter(); SafeCounter s = new SafeCounter();
Runnable work = () -> { for (int i = 0; i < 100_000; i++) { u.inc(); s.inc(); } };
Thread[] ts = new Thread[8];
for (int i = 0; i < 8; i++) { ts[i] = new Thread(work); ts[i].start(); }
for (Thread t : ts) t.join();
System.out.println("unsafe=" + u.count + " safe=" + s.count.get());   // unsafe varies every run
```

▶ Output

```
unsafe=<varies; below 800000 on multi-core CPUs> safe=800000
```

💡 Pro tip

The cheapest thread safety is not sharing: keep state method-local, make shared objects immutable, and share data by passing messages through queues.

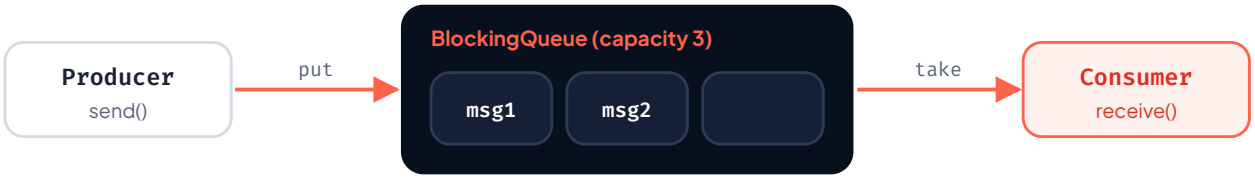
Q144 JP-023 • Advanced

How do threads communicate? Explain `wait()`, `notify()` and `notifyAll()`.

⚡ Quick answer

Inside a `synchronized` block, `wait()` releases the lock and suspends the thread until another thread calls `notify()` / `notifyAll()` on the same object. It is the low-level basis of producer-consumer coordination.

© TechTribers. For personal use only. Do not redistribute.



Full → producer waits. Empty → consumer waits. wait()/notifyAll() is the low-level version of the same idea.

Java

```java
class Mailbox {
    private String content;
    private boolean available;

    public synchronized void send(String msg) throws InterruptedException {
        while (available) wait();          // ALWAYS a loop: spurious wakeups exist
        content = msg; available = true;
        notifyAll();
    }
    public synchronized String receive() throws InterruptedException {
        while (!available) wait();
        available = false;
        notifyAll();
        return content;
    }
}

Mailbox box = new Mailbox();
Thread producer = new Thread(() -> {
    try { for (String m : List.of("order-1", "order-2", "DONE")) box.send(m); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
producer.start();
String msg;
while (!(msg = box.receive()).equals("DONE")) System.out.println("Got " + msg);
```

▶ Output

```
Got order-1
Got order-2
```

⚠️ **Interview trap**

Calling `wait()` / `notify()` without holding the object's lock throws `IllegalMonitorStateException`. Using `if` instead of `while` around `wait()` breaks on spurious wakeups.

💡 **Pro tip**

In real code use `BlockingQueue` (`put` / `take`), `CountDownLatch`, `CompletableFuture` or `Condition` from `java.util.concurrent`. They do the same job without the footguns.

© TechTribers. For personal use only. Do not redistribute.

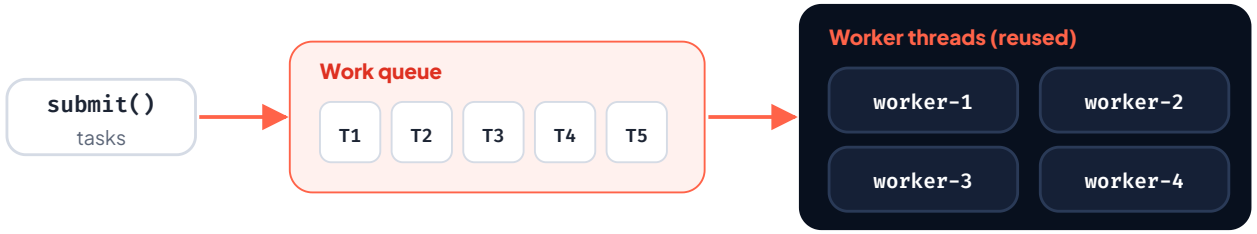
Q145 JP-024

• Intermediate

What is a thread pool? How do you use ExecutorService?

⚡ Quick answer

A thread pool keeps a set of reusable worker threads and a task queue. You submit tasks to an `ExecutorService` instead of creating threads, which bounds resource usage and avoids thread-creation cost.



core size · max size · queue · rejection policy are the four knobs of ThreadPoolExecutor.

Java

```
ExecutorService pool = new ThreadPoolExecutor(
        4, 8,                                       // core, max threads
        60, TimeUnit.SECONDS,                       // idle timeout for extra threads
        new ArrayBlockingQueue<>(100),              // BOUNDED queue
        new ThreadPoolExecutor.CallerRunsPolicy());  // backpressure when full

List<Future<Integer>> results = new ArrayList<>();
for (int i = 1; i <= 5; i++) {
    int n = i;
    results.add(pool.submit(() -> n * n));          // Callable<Integer>
}
int sum = 0;
for (Future<Integer> f : results) sum += f.get();
pool.shutdown();
pool.awaitTermination(5, TimeUnit.SECONDS);
System.out.println("Sum of squares = " + sum);
```

▶ Output

```
Sum of squares = 55
```

⚠️ Interview trap

`Executors.newFixedThreadPool(n)` uses an **unbounded** queue. Under load, tasks pile up until `OutOfMemoryError`. Production code should configure `ThreadPoolExecutor` with a bounded queue and a rejection policy.

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

How many threads? CPU-bound work: about the number of cores. I/O-bound work: more threads, or on Java 21+ virtual threads (`Executors.newVirtualThreadPerTaskExecutor()`), where pooling is unnecessary.

Section 06

Networking, JDBC & Runtime Loading

TCP vs UDP, talking to databases safely, Swing layouts, and loading classes at runtime.

Questions 146–151 · 6 questions



📡 **Q146** JP-025 • Intermediate

What is the difference between TCP and UDP? How do you use UDP in Java?

⚡ **Quick answer**

TCP (`Socket` / `ServerSocket`) is connection-oriented, reliable and ordered. UDP (`DatagramSocket`) sends independent packets with no connection, no delivery or order guarantee, and lower latency.

	TCP	UDP
Connection	Handshake first	None
Reliability	Retransmits, ordered	Packets may be lost or reordered
Unit	Byte stream	Datagram (max ~64 KB)
Used by	HTTP/1.1, HTTP/2, JDBC, Kafka	DNS, video calls, gaming, metrics (StatsD), QUIC/HTTP3

TCP (Socket)

```
SYN →
← SYN-ACK
ACK →
data 1,2,3 → (ordered, ACKed)
```

reliable · ordered · connection

UDP (DatagramSocket)

```
packet 1 →
packet 3 →
packet 2 → (lost?)
no handshake, no retries
```

fast · no guarantees · datagrams

© TechTribers. For personal use only. Do not redistribute.

```java
// UDP receiver
try (DatagramSocket socket = new DatagramSocket(9876)) {
    byte[] buf = new byte[1024];
    DatagramPacket packet = new DatagramPacket(buf, buf.length);
    socket.receive(packet);                                    // blocks for ONE datagram
    System.out.println(new String(packet.getData(), 0, packet.getLength(), UTF_8));
}

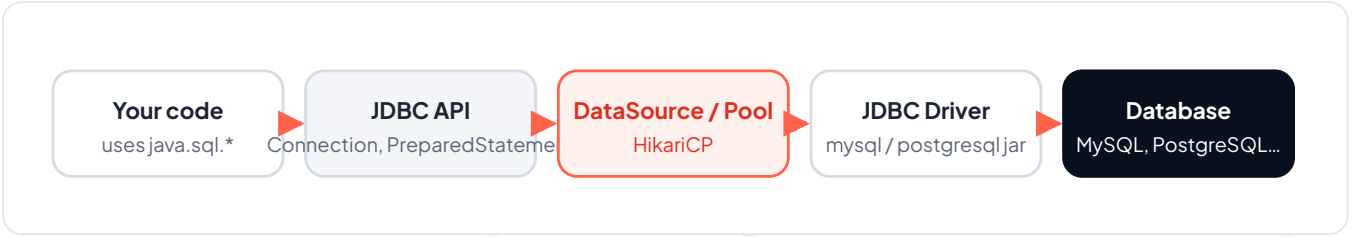
// UDP sender
try (DatagramSocket socket = new DatagramSocket()) {
    byte[] data = "metric cpu=73".getBytes(UTF_8);
    socket.send(new DatagramPacket(data, data.length, InetAddress.getByName("localhost"), 9876));
}
```

Q147 JP-026 • Basic

What is JDBC? How do you connect to a database?

> ⚡ **Quick answer**
> JDBC is Java's standard API (`java.sql`) for relational databases. You get a `Connection` (from `DriverManager` or, in real apps, a pooled `DataSource`), run statements, and read `ResultSet` s, closing everything with try-with-resources.

Since JDBC 4.0 (Java 6), drivers on the classpath register themselves automatically, so `Class.forName("com.mysql.jdbc.Driver")` is no longer needed.




```java
String url = "jdbc:postgresql://localhost:5432/shop";
try (Connection con = DriverManager.getConnection(url, System.getenv("DB_USER"),
System.getenv("DB_PASS"));
     PreparedStatement ps = con.prepareStatement("SELECT id, name, price FROM product WHERE price <
?")) {
    ps.setBigDecimal(1, new BigDecimal("1000"));
    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            System.out.printf("%d %s %s%n", rs.getLong("id"), rs.getString("name"),
rs.getBigDecimal("price"));
        }
```


© TechTribers. For personal use only. Do not redistribute.

```
    }
}
```

⚠️ **Interview trap**

Hard-coding `root` / `password` in source code (as many tutorials do) leaks credentials into Git. Read them from environment variables or a secrets manager. And never open a new `DriverManager` connection per request in a server: use a connection pool such as HikariCP.

💉 Q148 JP-027

• Intermediate

What is the difference between Statement and PreparedStatement?

⚡ **Quick answer**

`Statement` sends SQL text as-is, so concatenated user input enables **SQL injection**. `PreparedStatement` sends SQL with `?` placeholders and binds values separately: injection-safe, type-safe, and its plan can be reused.

	Statement	PreparedStatement
SQL injection	Vulnerable	Safe
Parameters	String concatenation	`setString`, `setLong`, … typed
Performance	Parsed every time	Precompiled, plan cached
Batching	`addBatch(sql)`	`addBatch()` with bindings

Java

```java
String userInput = "x' OR '1'='1";

// VULNERABLE: returns every row
String sql = "SELECT * FROM users WHERE name = '" + userInput + "'";
// stmt.executeQuery(sql);  ->  WHERE name = 'x' OR '1'='1'

// SAFE: the input is only ever data
try (PreparedStatement ps = con.prepareStatement("SELECT * FROM users WHERE name = ?")) {
    ps.setString(1, userInput);                    // matches a user literally named x' OR '1'='1
    ResultSet rs = ps.executeQuery();
}

// Batch inserts: one round trip for many rows
try (PreparedStatement ins = con.prepareStatement("INSERT INTO audit(event) VALUES (?)")) {
    for (String e : List.of("login", "view", "logout")) { ins.setString(1, e); ins.addBatch(); }
    ins.executeBatch();
}
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Placeholders bind **values** only. Table names, column names and `ORDER BY` direction can't be parameters; validate them against an allow-list.

Q149 JP-028 • Intermediate

How do you manage transactions in JDBC?

⚡ **Quick answer**

Turn off auto-commit, run the statements, then `commit()` on success or `rollback()` on any failure, so the group is atomic. Restore auto-commit before returning the connection to the pool.

Isolation level is set with `con.setTransactionIsolation(...)`: `READ_COMMITTED` (common default), `REPEATABLE_READ`, `SERIALIZABLE`. Use `Savepoint` to roll back part of a transaction.




```java
static void transfer(DataSource ds, long from, long to, BigDecimal amt) throws SQLException {
    try (Connection con = ds.getConnection()) {
        con.setAutoCommit(false);
        try (PreparedStatement debit  = con.prepareStatement("UPDATE account SET balance = balance -
? WHERE id = ? AND balance >= ?");
             PreparedStatement credit = con.prepareStatement("UPDATE account SET balance = balance +
? WHERE id = ?")) {
            debit.setBigDecimal(1, amt); debit.setLong(2, from); debit.setBigDecimal(3, amt);
            if (debit.executeUpdate() != 1) throw new SQLException("Insufficient funds");
            credit.setBigDecimal(1, amt); credit.setLong(2, to);
            credit.executeUpdate();
            con.commit();
        } catch (SQLException e) {
            con.rollback();
            throw e;
        } finally {
            con.setAutoCommit(true);
        }
    }
}
```

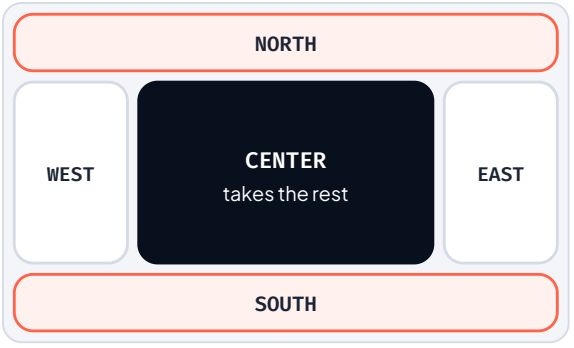

© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> In Spring, `@Transactional` does exactly this commit/rollback dance for you through a proxy. Knowing the raw JDBC version is what lets you debug it when it doesn't roll back (for example, on checked exceptions by default).

Q150 JP-029 • Basic

What are layout managers in Swing?

> ⚡ **Quick answer**
> A layout manager decides the position and size of components inside a container, so the UI adapts to window size, fonts and platforms instead of using fixed pixel coordinates.



`FlowLayout`	left → right, wraps
`BorderLayout`	5 regions (JFrame default)
`GridLayout`	equal cells, rows×cols
`BoxLayout`	single row or column
`GridBagLayout`	flexible grid, most control

```java
SwingUtilities.invokeLater(() -> {
    JFrame f = new JFrame("Login");
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));        // 2 rows x 2 cols
    form.add(new JLabel("User:"));     form.add(new JTextField(15));
    form.add(new JLabel("Password:")); form.add(new JPasswordField(15));

    JPanel buttons = new JPanel(new FlowLayout(FlowLayout.RIGHT));
    buttons.add(new JButton("Cancel")); buttons.add(new JButton("Login"));

    f.add(new JLabel("TechTribers", SwingConstants.CENTER), BorderLayout.NORTH);
    f.add(form, BorderLayout.CENTER);
    f.add(buttons, BorderLayout.SOUTH);
    f.pack(); f.setVisible(true);
});
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

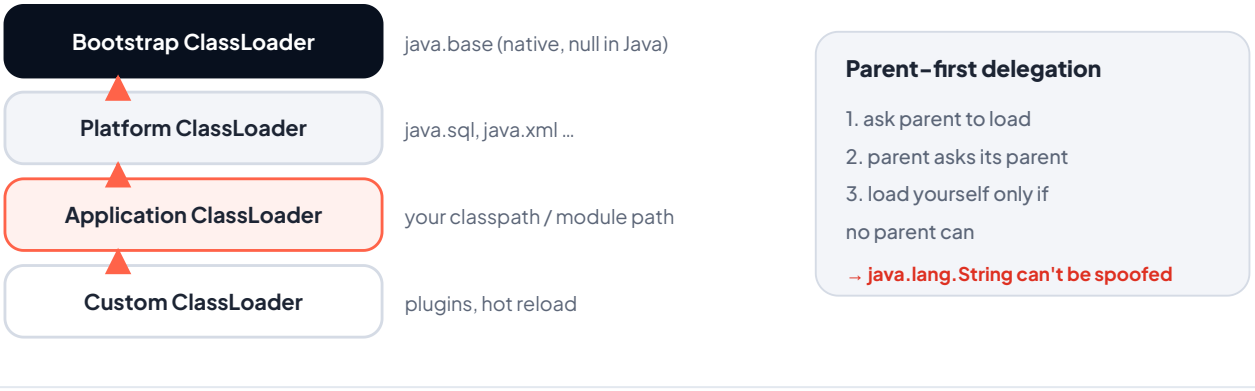
Real screens nest panels, each with a simple layout (as above), rather than fighting one giant `GridBagLayout`. `setLayout(null)` with absolute positions breaks on high-DPI screens and other fonts.

Q151 JP-030 • Advanced

What is dynamic class loading? How does `Class.forName()` work?

⚡ **Quick answer**

Classes can be loaded by name at runtime with `Class.forName("pkg.Name")` or a `ClassLoader`, then instantiated reflectively. This powers plugins, JDBC drivers, and frameworks that read class names from configuration.



Java

```java
// The implementation class name comes from config, not from code
String impl = "java.util.ArrayList";
Class<?> clazz = Class.forName(impl);                    // loads + initialises
@SuppressWarnings("unchecked")
List<String> list = (List<String>) clazz.getDeclaredConstructor().newInstance();
list.add("loaded dynamically");
System.out.println(clazz.getSimpleName() + ": " + list);

// The idiomatic, type-safe way: ServiceLoader (META-INF/services)
ServiceLoader<java.sql.Driver> drivers = ServiceLoader.load(java.sql.Driver.class);
System.out.println("Drivers found: " + drivers.stream().count());
```

▶ **Output**

```
ArrayList: [loaded dynamically]
Drivers found: 0
```

⚠️ **Interview trap**

`clazz.newInstance()` (used in many notes) has been deprecated since Java 9 because it sneakily rethrows checked exceptions from the constructor. Use `getDeclaredConstructor().newInstance()`.

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Why parent-first delegation? So application code can never replace core classes like `java.lang.String`. Web servers and OSGi deliberately use child-first loaders to isolate each application's libraries.

Section 07

Java 8 Functional Programming

Lambdas, the java.util.function toolkit, and the Stream API: the features that changed how modern Java is written.

Questions 152–154 · 3 questions

λ

λ **Q152** JP-031 • Basic

What are lambda expressions?

⚡ **Quick answer**

A lambda is a concise anonymous function, `(params) -> expression` or `-> { block }`, that implements a **functional interface**. Method references (`String::length`) are an even shorter form.

Lambdas can use local variables from the enclosing scope only if they are **effectively final**. Unlike anonymous classes, `this` inside a lambda refers to the enclosing instance, and no new class file is generated (they're linked via `invokedynamic`).

Java

```
List<String> names = new ArrayList<>(List.of("Ravi", "Asha", "Om"));

names.sort((a, b) -> a.compareTo(b));                // lambda
names.sort(Comparator.naturalOrder());               // same, via factory
names.forEach(System.out::println);                  // method reference

Runnable task = () -> System.out.println("no params");
BinaryOperator<Integer> add = (x, y) -> x + y;
Function<String, Integer> len = String::length;
task.run();
System.out.println(add.apply(2, 3) + " " + len.apply("TechTribers"));
```

▶ **Output**

```
Asha
Om
Ravi
no params
5 11
```


© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

Four kinds of method reference: static (`Integer::parseInt`), bound instance (`System.out::println`), unbound instance (`String::length`), and constructor (`ArrayList::new`).

Q153 JP-032 • Basic

What are functional interfaces? Name the common ones.

⚡ **Quick answer**

An interface with exactly **one abstract method** (default and static methods don't count). Any lambda or method reference can implement it. `@FunctionalInterface` makes the compiler enforce the rule.

Predicate T → boolean `test()`	**Function** T → R `apply()`	**Consumer** T → void `accept()`
Supplier () → T `get()`	**UnaryOperator** T → T `apply()`	**BiFunction** (T,U) → R `apply()`

Java

```
Predicate<Integer> isEven = n -> n % 2 == 0;
Function<String, Integer> length = String::length;
Consumer<String> shout = s -> System.out.println(s.toUpperCase());
Supplier<String> id = () -> "TT-" + 42;

System.out.println(isEven.and(n -> n > 10).test(12));      // composition
System.out.println(length.andThen(n -> n * 2).apply("Hello"));
shout.accept("java");
System.out.println(id.get());

@FunctionalInterface
interface DiscountRule { BigDecimal apply(BigDecimal price); }
DiscountRule festive = p -> p.multiply(new BigDecimal("0.8"));
System.out.println(festive.apply(new BigDecimal("500")));
```

▶ **Output**

```
true
10
JAVA
TT-42
400.0
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> Use the primitive specialisations (`IntPredicate` , `ToIntFunction` , `IntSupplier`) in hot paths to avoid boxing every value.

Q154 JP-033 • Intermediate

What is the Stream API? How do you use it?

> ⚡ **Quick answer**
> A stream is a lazy pipeline over a data source: a source, zero or more **intermediate** operations (`filter` , `map` , `sorted`), and one **terminal** operation (`collect` , `sum` , `forEach`) that triggers execution.

Intermediate ops are lazy; nothing runs until the terminal op.



Elements flow one at a time through the whole chain (loop fusion), not stage by stage.

Java

```
record Order(String customer, String city, double amount) { }
List<Order> orders = List.of(new Order("Asha", "Pune", 1200), new Order("Ravi", "Mumbai", 450),
                             new Order("Om", "Pune", 800), new Order("Asha", "Pune", 300));

double puneTotal = orders.stream()
        .filter(o -> o.city().equals("Pune"))
        .mapToDouble(Order::amount)
        .sum();

Map<String, Double> byCustomer = orders.stream()
        .collect(Collectors.groupingBy(Order::customer, TreeMap::new,
Collectors.summingDouble(Order::amount)));

List<String> bigSpenders = orders.stream()
        .filter(o -> o.amount() > 500).map(Order::customer).distinct().sorted().toList();   //
toList(): Java 16

System.out.println(puneTotal + " | " + byCustomer + " | " + bigSpenders);
```

▶ **Output**

```
2300.0 | {Asha=1500.0, Om=800.0, Ravi=450.0} | [Asha, Om]
```

> ⚠️ **Interview trap**
> A stream can be consumed only **once**; reusing it throws `IllegalStateException` . And `parallel()` is not a free speed-up: it uses the shared common ForkJoinPool and often slows down small or I/O-bound workloads.

© TechTribers. For personal use only. Do not redistribute.

🎯 **Likely follow-up**

`map` vs `flatMap`? `map` transforms each element 1-to-1; `flatMap` turns each element into a stream and flattens the result, for example `orders.stream().flatMap(o -> o.items().stream())`.

Section 08

Design Patterns

The patterns interviewers ask about most, each with a production-style Java example and where you've already used it.

Questions 155–164 · 10 questions



Q155 JP-034

• Basic

What are design patterns? What are the main categories?

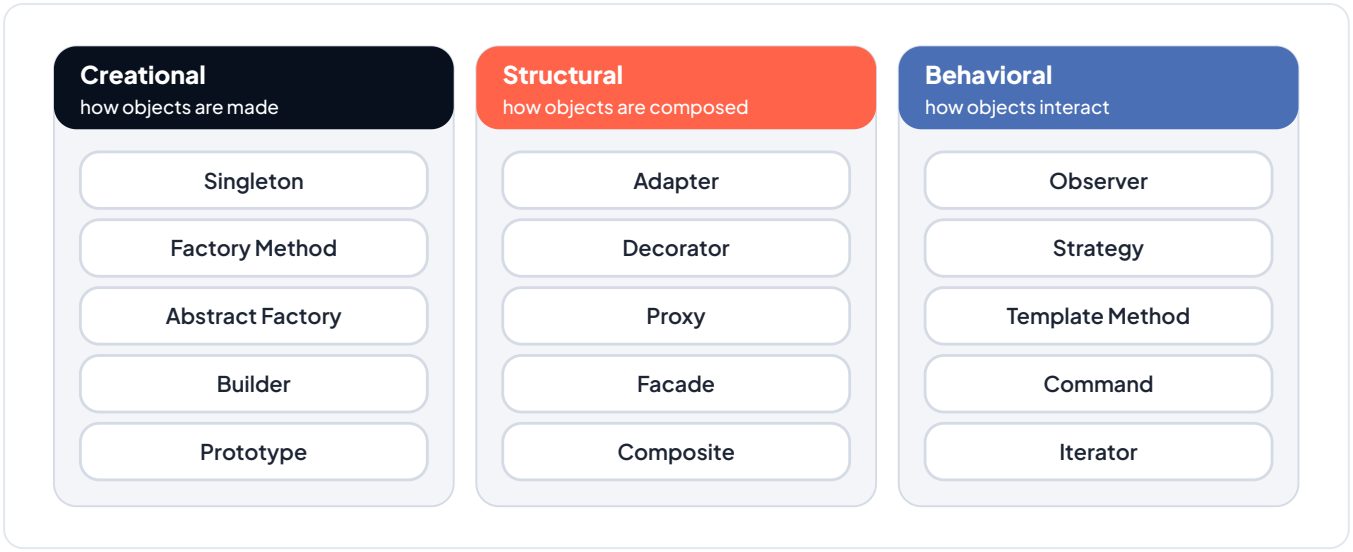
⚡ **Quick answer**

Design patterns are named, reusable solutions to recurring design problems, catalogued by the "Gang of Four" (1994). They fall into three groups: **creational** (making objects), **structural** (composing objects) and **behavioral** (how objects interact).

Patterns you have already used in the JDK and Spring:

- **Singleton**: Spring beans (singleton scope), `Runtime.getRuntime()`.
- **Factory**: `List.of()`, `Calendar.getInstance()`, `BeanFactory`.
- **Builder**: `StringBuilder`, `HttpRequest.newBuilder()`, Lombok `@Builder`.
- **Decorator**: `BufferedReader(new FileReader(...))`.
- **Proxy**: Spring AOP, `@Transactional`, Hibernate lazy loading.
- **Observer**: `ApplicationEventPublisher`, Kafka topics.
- **Strategy**: `Comparator`, Spring `PasswordEncoder`.
- **Template Method**: `JdbcTemplate`, `AbstractList`.

© TechTribers. For personal use only. Do not redistribute.



💡 **Pro tip**

In interviews, don't just define a pattern: name the **problem** it solves and a place it appears in code you've worked on. That's what separates memorisation from experience.

1️⃣ **Q156** JP-035 • Intermediate

What is the Singleton pattern? How do you implement it correctly?

⚡ **Quick answer**

Singleton guarantees one instance of a class with a global access point. The safest Java implementations are an **enum** or a **lazy holder** class; double-checked locking needs `volatile` to be correct.

Java

```java
// 1. Enum: thread-safe, serialization-safe, reflection-safe (Effective Java, Item 3)
public enum AppConfig {
    INSTANCE;
    private final Map<String, String> props = Map.of("region", "ap-south-1");
    public String get(String k) { return props.get(k); }
}

// 2. Lazy holder: created on first getInstance(), no locking cost
public final class IdGenerator {
    private IdGenerator() { }
    private static class Holder { static final IdGenerator INSTANCE = new IdGenerator(); }
    public static IdGenerator getInstance() { return Holder.INSTANCE; }
}

// 3. Double-checked locking: volatile is REQUIRED
public final class Registry {
    private static volatile Registry instance;
    private Registry() { }
```


© TechTribers. For personal use only. Do not redistribute.

```
    public static Registry getInstance() {
        Registry r = instance;
        if (r == null) {
            synchronized (Registry.class) {
                r = instance;
                if (r == null) instance = r = new Registry();
            }
        }
        return r;
    }
}
```

⚠️ **Interview trap**

Double-checked locking **without** `volatile` is broken: another thread can see a non-null reference to a half-constructed object. Also, a private constructor alone doesn't stop reflection or deserialization from creating a second instance; the enum form does.

🎯 **Likely follow-up**

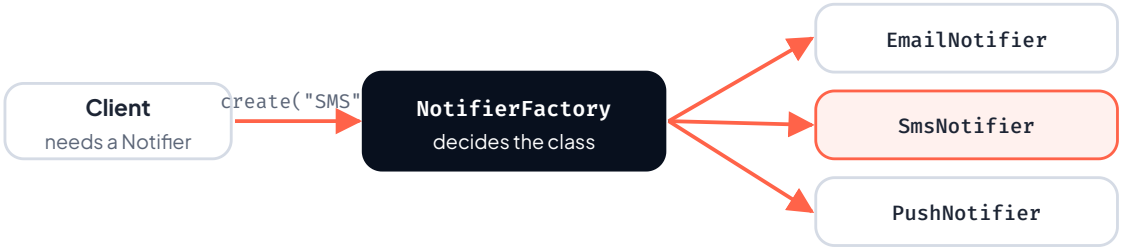
Why is Singleton often called an anti-pattern? Global mutable state hides dependencies and makes unit tests hard. Prefer dependency injection with a singleton-scoped bean.

Q157 JP-036 • Intermediate

What is the Factory pattern? Factory Method vs Abstract Factory?

⚡ **Quick answer**

A factory hides object creation behind a method, so callers depend on an interface rather than concrete classes. **Factory Method**: one method decides which subtype to create. **Abstract Factory**: an object that creates a whole **family** of related objects.



Client depends only on the Notifier interface, never on concrete classes.

Java

```
interface Notifier { void send(String to, String msg); }
class EmailNotifier implements Notifier { public void send(String t, String m) {
System.out.println("Email " + t + ": " + m); } }
class SmsNotifier   implements Notifier { public void send(String t, String m) {
System.out.println("SMS " + t + ": " + m); } }
```


© TechTribers. For personal use only. Do not redistribute.

```
final class NotifierFactory {                                  // factory method
    static Notifier create(String channel) {
        return switch (channel) {
            case "EMAIL" -> new EmailNotifier();
            case "SMS"   -> new SmsNotifier();
            default      -> throw new IllegalArgumentException("Unknown channel " + channel);
        };
    }
}

// Abstract factory: a family that must match (e.g. per cloud provider)
// interface CloudFactory { Storage storage(); MessageQueue queue(); }
// class AwsFactory implements CloudFactory { S3Storage, SqsQueue }
// class AzureFactory implements CloudFactory { BlobStorage, ServiceBusQueue }

NotifierFactory.create("SMS").send("+91-98xxxx", "OTP 4821");
```

▶ Output

```
SMS +91-98xxxx: OTP 4821
```

💡 **Pro tip**

In Spring, the container is the factory: inject `Map<String, Notifier>` and Spring fills it with every `Notifier` bean keyed by bean name. Adding a channel then needs no factory change at all.

Q158 JP-037 • Intermediate

What is the Builder pattern and when should you use it?

⚡ **Quick answer**

Builder constructs a complex object step by step through a fluent API, then validates and creates it in `build()`. Use it when a class has many optional parameters or must be immutable.

It replaces the **telescoping constructor** problem (`new HttpCall(url, null, null, 3, true, null)`), where nobody can tell which argument is which.

Java

```
public final class HttpCall {
    private final String url; private final String method; private final Duration timeout; private final Map<String, String> headers;

    private HttpCall(Builder b) { url = b.url; method = b.method; timeout = b.timeout; headers = Map.copyOf(b.headers); }
    public static Builder to(String url) { return new Builder(url); }
    @Override public String toString() { return method + " " + url + " " + timeout.toSeconds() + "s " + headers; }

    public static final class Builder {
        private final String url; private String method = "GET";
```


© TechTribers. For personal use only. Do not redistribute.

```
        private Duration timeout = Duration.ofSeconds(10); private final Map<String, String> headers
= new TreeMap<>();
        private Builder(String url) { this.url = Objects.requireNonNull(url); }
        public Builder post() { method = "POST"; return this; }
        public Builder timeout(Duration d) { timeout = d; return this; }
        public Builder header(String k, String v) { headers.put(k, v); return this; }
        public HttpCall build() {
            if (timeout.isNegative()) throw new IllegalStateException("timeout < 0");
            return new HttpCall(this);
        }
    }
}

System.out.println(HttpCall.to("https://api.techtriber.com/orders").post()
        .header("X-Trace", "abc").timeout(Duration.ofSeconds(3)).build());
```

▶ Output

```
POST https://api.techtriber.com/orders 3s {X-Trace=abc}
```

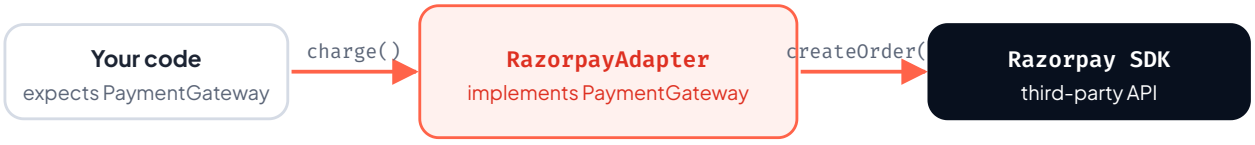
Q159 JP-038 • Intermediate

What is the Adapter pattern?

⚡ Quick answer

Adapter wraps an existing class to make it fit the interface your code expects, so incompatible interfaces can work together without changing either side.

JDK examples: `Arrays.asList()` adapts an array to `List`; `InputStreamReader` adapts a byte stream to a character `Reader`.



Translate one interface into another without changing either side. Like a travel plug adapter.

Java

```
interface PaymentGateway { String charge(String orderId, long paise); }      // what our code uses

class LegacyBankApi {                                                          // can't change it
    String makePayment(double rupees, String ref) { return "TXN-" + ref + "-" + rupees; }
}

class LegacyBankAdapter implements PaymentGateway {
    private final LegacyBankApi api = new LegacyBankApi();
    @Override public String charge(String orderId, long paise) {
```


© TechTribers. For personal use only. Do not redistribute.

```
        return api.makePayment(paise / 100.0, orderId);                    // translate units +
names
    }
}

PaymentGateway gw = new LegacyBankAdapter();
System.out.println(gw.charge("ORD42", 49_900));
```

▶ Output

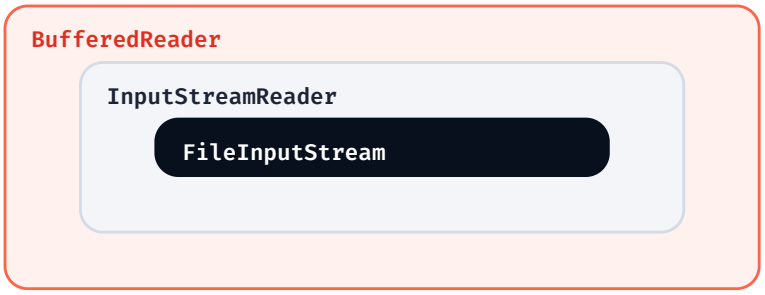
```
TXN-ORD42-499.0
```

Q160 JP-039 • Intermediate

What is the Decorator pattern?

⚡ Quick answer

Decorator adds behaviour to an object at runtime by wrapping it in another object with the **same interface**. Decorators can be stacked, giving flexible combinations without a subclass for every mix.



Each layer wraps the next and adds one responsibility:

`buffering → char decoding → bytes`

Same interface outside, extra behaviour inside.

Java

```
interface Notifier { String send(String msg); }
record BaseNotifier() implements Notifier { public String send(String m) { return m; } }

record Timestamped(Notifier inner) implements Notifier {
    public String send(String m) { return inner.send("[09:30] " + m); }
}
record Encrypted(Notifier inner) implements Notifier {
    public String send(String m) { return inner.send(new StringBuilder(m).reverse().toString()); }
}

Notifier n = new Encrypted(new Timestamped(new BaseNotifier()));   // stack in any order
System.out.println(n.send("hello"));
```

▶ Output

```
[09:30] olleh
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

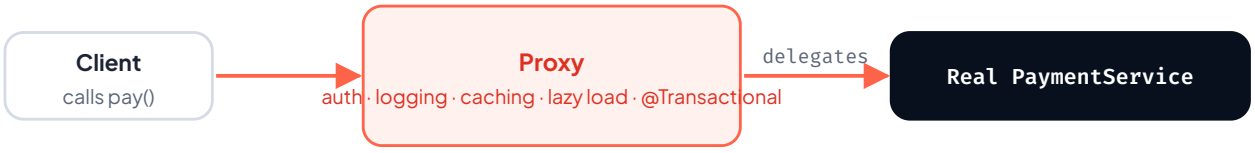
Decorator vs inheritance: inheritance fixes behaviour at compile time and explodes into classes like `TimestampedEncryptedNotifier`. Decorators compose at runtime. Java I/O streams are the textbook example.

🕵️ **Q161** JP-040 • Intermediate

What is the Proxy pattern?

⚡ **Quick answer**

A proxy stands in for a real object with the same interface and controls access to it: adding security checks, caching, logging, lazy loading or remote calls before delegating.



Spring AOP creates JDK dynamic proxies (interfaces) or CGLIB subclasses (classes) around your beans.

Java

```
interface PriceService { BigDecimal price(String sku); }
class SlowPriceService implements PriceService {
    public BigDecimal price(String sku) { System.out.println("DB hit " + sku); return new
BigDecimal("499"); }
}

// Dynamic proxy: one handler wraps ANY interface
@SuppressWarnings("unchecked")
static <T> T caching(T target, Class<T> type) {
    Map<String, Object> cache = new ConcurrentHashMap<>();
    return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, (p, m, args) ->
            cache.computeIfAbsent(m.getName() + Arrays.toString(args), k -> {
                try { return m.invoke(target, args); } catch (Exception e) { throw new
RuntimeException(e); }
            }));
}

PriceService ps = caching(new SlowPriceService(), PriceService.class);
System.out.println(ps.price("SKU-1") + " " + ps.price("SKU-1"));
```

▶ **Output**

```
DB hit SKU-1
499 499
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

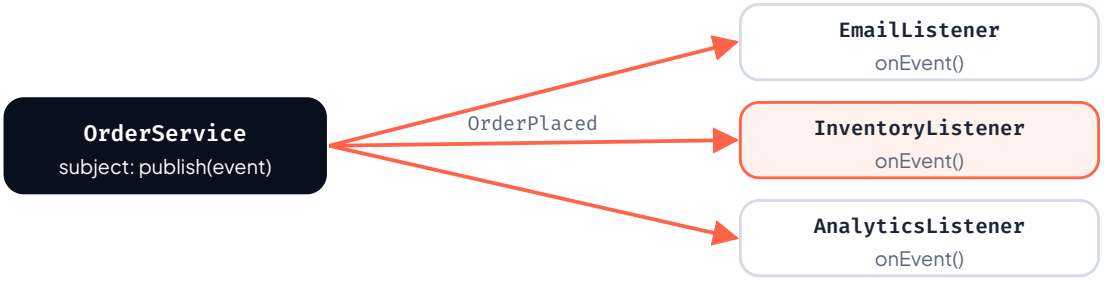
Spring's proxy-based `@Transactional` and `@Cacheable` don't apply when a bean calls its own method (`this.save()`), because the call never goes through the proxy. This self-invocation gotcha is a favourite senior-level question.

📢 **Q162** JP-041 • Intermediate

What is the Observer pattern?

⚡ **Quick answer**

Observer defines a one-to-many dependency: when a subject's state changes, all registered observers are notified automatically, without the subject knowing their concrete types.



One-to-many: the subject doesn't know who listens. Kafka topics are this pattern at system scale.

Java

```
record OrderPlaced(String orderId, double amount) { }

class EventBus {
    private final List<Consumer<OrderPlaced>> listeners = new CopyOnWriteArrayList<>();
    void subscribe(Consumer<OrderPlaced> l) { listeners.add(l); }
    void publish(OrderPlaced e) { listeners.forEach(l -> l.accept(e)); }
}

EventBus bus = new EventBus();
bus.subscribe(e -> System.out.println("Email receipt for " + e.orderId()));
bus.subscribe(e -> System.out.println("Reserve stock for " + e.orderId()));
bus.subscribe(e -> { if (e.amount() > 1000) System.out.println("Fraud check " + e.orderId()); });

bus.publish(new OrderPlaced("ORD-7", 2500));
```

▶ **Output**

```
Email receipt for ORD-7
Reserve stock for ORD-7
Fraud check ORD-7
```


© TechTribers. For personal use only. Do not redistribute.

> 💡 **Pro tip**
> In Spring use `ApplicationEventPublisher` and `@EventListener` (or `@TransactionalEventListener` to fire only after commit). Across services, the same pattern becomes Kafka topics with consumer groups.

Q163 JP-042 • Intermediate

What is the Strategy pattern?

> ⚡ **Quick answer**
> Strategy defines a family of interchangeable algorithms behind one interface and lets the caller choose one at runtime, replacing long `if/else` or `switch` chains.

`Comparator` is the strategy you use every day: `list.sort(byPrice)` vs `list.sort(byName)` swaps the comparison algorithm without changing `sort`.



Swap the algorithm at runtime without touching Checkout (Open/Closed principle).

```java
@FunctionalInterface
interface PricingStrategy { double finalPrice(double base); }

enum Pricing implements PricingStrategy {
    REGULAR  { public double finalPrice(double b) { return b; } },
    FESTIVAL { public double finalPrice(double b) { return b * 0.80; } },
    PRIME    { public double finalPrice(double b) { return b * 0.90; } }
}

record Checkout(PricingStrategy strategy) {
    double total(List<Double> items) { return strategy.finalPrice(items.stream().mapToDouble(d ->
d).sum()); }
}

List<Double> cart = List.of(1000.0, 500.0);
for (Pricing p : Pricing.values()) System.out.println(p + " -> " + new Checkout(p).total(cart));
System.out.println("custom -> " + new Checkout(b -> b - 100).total(cart));   // lambda strategy
```

▶ **Output**

```
REGULAR -> 1500.0
FESTIVAL -> 1200.0
```


© TechTribers. For personal use only. Do not redistribute.

```
PRIME -> 1350.0
custom -> 1400.0
```

Q164 JP-043

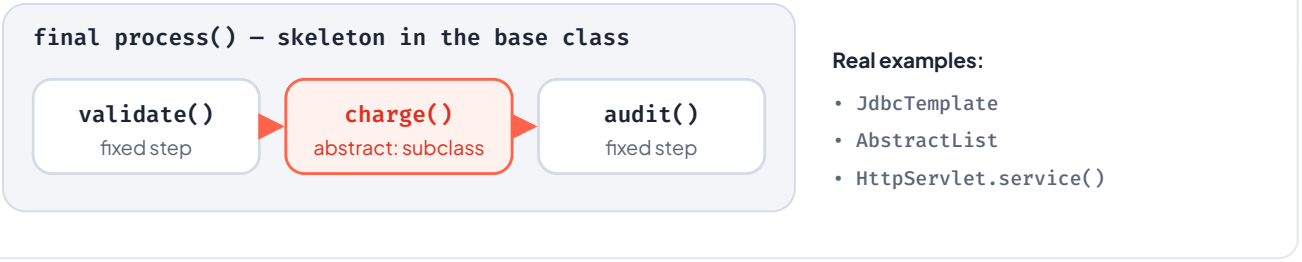
• Intermediate

What is the Template Method pattern?

⚡ **Quick answer**

Template Method puts an algorithm's fixed skeleton in a base class method (often `final`) and lets subclasses fill in specific steps by overriding abstract or hook methods.

Template Method uses **inheritance** to vary steps; Strategy uses **composition**. Modern code often prefers passing lambdas (strategy) over subclassing, but `JdbcTemplate`, `AbstractList` and `HttpServlet` are classic template methods.



Java

```java
abstract class ReportJob {
    public final void run() {                    // the template: order is fixed
        var rows = fetch();
        var lines = rows.stream().map(this::format).toList();
        publish(lines);
    }
    protected abstract List<String> fetch();
    protected String format(String row) { return "- " + row; }      // hook with default
    protected void publish(List<String> lines) { lines.forEach(System.out::println); }
}

class SalesReport extends ReportJob {
    protected List<String> fetch() { return List.of("Pune 4.2L", "Mumbai 3.1L"); }
    @Override protected String format(String row) { return "₹ " + row; }
}

new SalesReport().run();
```

▶ **Output**

```
₹ Pune 4.2L
₹ Mumbai 3.1L
```


© TechTribers. For personal use only. Do not redistribute.

Section 09

Best Practices, Testing & Debugging

Conventions, exception and memory hygiene, performance work, JUnit 5, and a repeatable way to debug.

Questions 165–170 · 6 questions



Q165 JP-044

• Basic

What are Java naming conventions and code-formatting best practices?

⚡ Quick answer

Classes and interfaces in PascalCase, methods and variables in camelCase, constants in UPPER_SNAKE_CASE, packages in lowercase reverse-domain form. Format consistently and let a tool enforce it.

Element	Convention	Example
Class / interface / record	PascalCase noun	`OrderService`, `PaymentGateway`
Method	camelCase verb	`calculateTotal()`, `isActive()`
Variable	camelCase, meaningful	`retryCount`, not `rc`
Constant	UPPER_SNAKE_CASE	`MAX_RETRIES`
Package	lowercase, reverse domain	`com.techtribers.billing`
Type parameter	single capital	`T`, `K`, `V`, `E`
Boolean	`is` / `has` / `can` prefix	`isEnabled`, `hasStock`

💡 Pro tip

Stop arguing formatting in code review: adopt Google Java Format or Palantir Java Format with Spotless in the Maven/Gradle build, and Checkstyle or Sonar for the rest. Always use braces, even for one-line `if` bodies.

© TechTribers. For personal use only. Do not redistribute.

Q166 JP-045

• Intermediate

What are the best practices for exception handling?

> ⚡ **Quick answer**
> Catch specific exceptions, handle them at the level that can actually recover, never swallow them, preserve the cause when wrapping, and close resources with try-with-resources.

Checklist:

- Don't use exceptions for normal control flow; they are expensive to create (stack trace capture).
- Throw early (validate inputs), catch late (where you can do something useful).
- Prefer unchecked custom exceptions for business errors in modern code; use checked ones only when the caller can genuinely recover.
- Never log **and** rethrow the same exception at every layer: it produces duplicate stack traces.
- Restore the interrupt flag when catching `InterruptedException`: `Thread.currentThread().interrupt()`.

```java
// ✗ Bad: swallows the error, loses the cause, catches everything
try { repo.save(order); } catch (Exception e) { System.out.println("error"); }

// ✓ Good
try {
    repo.save(order);
} catch (DataIntegrityViolationException e) {                    // specific
    throw new DuplicateOrderException(order.id(), e);            // wrap WITH cause
}

// ✓ Log once, at the boundary (e.g. @ControllerAdvice), with context
log.error("Payment failed orderId={} amount={}", order.id(), order.amount(), e);
```

Q167 JP-046

• Advanced

How do memory leaks happen in Java, and how do you prevent them?

> ⚡ **Quick answer**
> The GC frees only **unreachable** objects. A leak happens when objects stay reachable but unused: static collections that only grow, unremoved listeners, `ThreadLocal`s in thread pools, unclosed resources, and inner-class references.

Common culprits and fixes:

- **Unbounded static caches** → use a bounded cache (Caffeine) with size or time limits.
- `ThreadLocal` **in pooled threads** → always `remove()` in a `finally` block.
- **Listeners/callbacks never deregistered** → unregister them, or use weak references.
- **Unclosed streams, connections,** `ResultSet`**s** → try-with-resources.

© TechTribers. For personal use only. Do not redistribute.

- **Mutable `HashMap` keys** → a key's hash changes and the entry becomes unreachable but stays in the map.




```
Terminal
// Diagnose
// 1. Watch heap after full GC trend upward:   jcmd <pid> GC.heap_info
// 2. Dump the heap:                           jcmd <pid> GC.heap_dump /tmp/heap.hprof
// 3. Open in Eclipse MAT -> "Leak Suspects", check dominator tree
// 4. Always run with: -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dumps
```

⚠️ **Interview trap**

Calling `System.gc()` or setting variables to `null` everywhere doesn't fix leaks. If something still references the object, it stays.

🚀 **Q168** JP-047 • Advanced

How do you approach Java performance optimization?

⚡ **Quick answer**

Measure first, then fix the biggest bottleneck: profile with JFR/async-profiler, benchmark with JMH, and focus on algorithms, I/O, allocation and contention before micro-tweaks.

A practical order of attack:

- **Measure**: Java Flight Recorder (`-XX:StartFlightRecording`), async-profiler flame graphs, APM traces. Never guess.
- **I/O first**: N+1 queries, missing indexes, chatty remote calls, no connection pooling. These usually dominate.
- **Algorithms and data structures**: O(n²) loops, `List.contains` inside loops (use a `HashSet`), wrong collection type.
- **Allocation**: string concatenation in loops, boxing in hot paths, huge temporary collections.
- **Concurrency**: lock contention, undersized pools, blocking calls on event-loop threads. Consider virtual threads for blocking I/O.
- **JVM tuning last**: right GC (G1 default, ZGC for low latency), heap size, container-aware settings.

© TechTribers. For personal use only. Do not redistribute.

```java
@BenchmarkMode(Mode.AverageTime) @OutputTimeUnit(TimeUnit.NANOSECONDS)
@State(Scope.Benchmark)
public class LookupBench {
    List<Integer> list = IntStream.range(0, 10_000).boxed().toList();
    Set<Integer> set = new HashSet<>(list);
    @Benchmark public boolean listContains() { return list.contains(9_999); }   // O(n)
    @Benchmark public boolean setContains()  { return set.contains(9_999); }    // O(1)
}
```

⚠️ **Interview trap**

Timing code with `System.currentTimeMillis()` around a loop measures JIT warm-up, dead-code elimination and GC noise. Use JMH for micro-benchmarks.

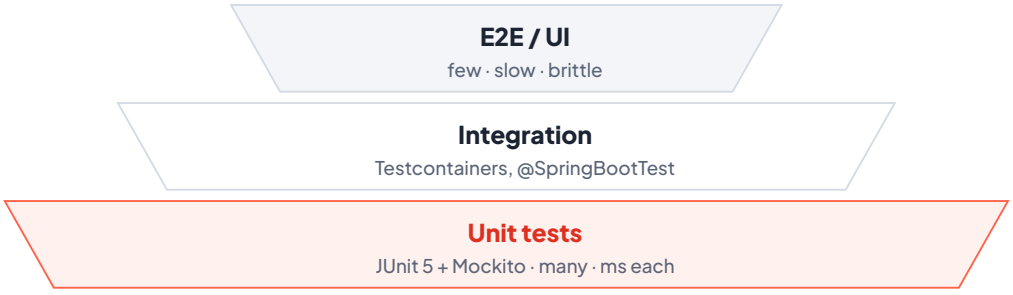
🧪 **Q169** JP-048 • Basic

How do you write unit tests with JUnit 5?

⚡ **Quick answer**

JUnit 5 (Jupiter) tests are methods annotated with `@Test`, organised with lifecycle hooks (`@BeforeEach`), assertions (`assertEquals`, `assertThrows`) and parameterized tests. Mockito replaces dependencies with mocks.

Good unit tests are **FIRST**: Fast, Isolated, Repeatable, Self-validating, Timely. Follow Arrange-Act-Assert, test one behaviour per test, and name tests after the behaviour, not the method.




```java
class PriceCalculatorTest {

    private PriceCalculator calc;

    @BeforeEach
    void setUp() { calc = new PriceCalculator(new BigDecimal("0.18")); }   // 18% GST

    @Test
```


© TechTribers. For personal use only. Do not redistribute.

```
    @DisplayName("adds GST to the base price")
    void addsGst() {
        assertEquals(new BigDecimal("118.00"), calc.withTax(new BigDecimal("100.00")));
    }

    @Test
    void rejectsNegativePrice() {
        var ex = assertThrows(IllegalArgumentException.class, () -> calc.withTax(new
BigDecimal("-1")));
        assertTrue(ex.getMessage().contains("negative"));
    }

    @ParameterizedTest
    @CsvSource({"0, 0.00", "50, 59.00", "1000, 1180.00"})
    void manyValues(BigDecimal in, BigDecimal expected) {
        assertEquals(expected, calc.withTax(in).setScale(2));
    }
}
```

⚠️ **Interview trap**

`assertEquals(new BigDecimal("118.0"), new BigDecimal("118.00"))` fails, because `BigDecimal.equals` compares scale too. Use `compareTo` (or AssertJ's `isEqualByComparingTo`) for money.

Q170 JP-049 • Intermediate

What debugging techniques and tools do you use?

⚡ **Quick answer**

Reproduce first, then narrow down: read the full stack trace, use the IDE debugger (breakpoints, conditional breakpoints, watches, evaluate expression), and in production rely on structured logs, thread dumps, heap dumps and JFR.

Symptom	Tool / technique
Exception	Read the **bottom** `Caused by:` first; it's the root cause
Wrong value	Conditional breakpoint, watch expression, evaluate expression
Hang / deadlock	`jstack <pid>` or `jcmd <pid> Thread.print` (2–3 dumps, a few seconds apart)
High CPU	`top -H` for the thread, match it in the thread dump; async-profiler
Memory growth / OOM	Heap dump + Eclipse MAT
Prod only	Structured logs with trace IDs, JFR, remote debug (staging only)

© TechTribers. For personal use only. Do not redistribute.

Terminal

```
# Capture evidence from a running JVM (no restart needed)
jcmd <pid> Thread.print > threads-1.txt
jcmd <pid> GC.heap_info
jcmd <pid> JFR.start duration=60s filename=/tmp/app.jfr
# Remote debugging (never in production)
java -agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=*:5005 -jar app.jar
```

💡 **Pro tip**

Once you've found a bug, write a failing test that reproduces it before fixing. The test proves the fix and stops the bug from coming back.

Series progress

170 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 4 starts at Q171.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.